Secure and Practical Endpoint Address Merging

ISCA 2026 Submission **#NaN** – Confidential Draft – Do NOT Distribute!!

***Abstract*—**

Historically, identical code and data share the same pages in memory. But it is later discovered that memory sharing enables a reuse-based side channel that can leak private/non-shared information. This causes data centers to disable memory sharing across jobs (e.g., different VMs, microVMs, containers).

To securely restore memory sharing, prior works have proposed many hardware defenses. But they modify many hardware components across the CPU, such as L1 caches, L2 caches, LLC, coherence directory, MMU, and the memory controller, making practical adoption and deployment difficult.

This paper proposes a practical hardware-based solution that reduces the total required number of hardware components to modify for achieving comprehensive protection against both cache and memory reuse-based side channels down to just one - the memory controller. This is achieved by enhancing the memory controller to eliminate all reuse-based side channels from caches, without any modification to caches. We prototype our design on a real system to verify whether it is indeed practical. Our real-system evaluations show it can securely restore memory sharing.

I. Introduction

Much contents in memory can be identical. For example, different users' jobs often rely on commonly used ***code***, such as open-source C and Python libraries, cryptography libraries, OS kernels, and utilities; meanwhile, the same user's different jobs may have large volumes of identical ***data*** in memory. Historically, identical code and data use shared pages in memory; the OS records the same physical page number (PPN) in the page table entries of virtual pages with identical content.

Unfortunately, it is later discovered that memory sharing opens up **reuse-based side channels** that can expose a victim's access behavior, which in turn can leak private information.

In caches, sharing physical addresses across jobs means they can access the same cachelines. Victim accesses to cachelines shared with the attacker can result in latency differences that are observable by attackers; this leads to various attacks [14], [15], [23], [24], [33], [37], [65]–[67], [69]. For example, in a Flush+Reload attack [65], the attacker first evicts shared code from the cache and later re-accesses it. The attacker can detect that the victim recently accessed the shared code by observing cache hit latency when re-accessing the code. If the victim's access depends on secret data, the attacker can infer sensitive information such as cryptographic keys [65].

In memory, sharing physical addresses across jobs means that after a victim accesses a memory location, an attacker can observe a row buffer hit (instead of a miss) when later accessing the same memory location, creating a reuse side channel in memory [48].

Together, these reuse-based side channels pose a severe security risk. As a result, cloud providers have disabled the sharing of memory across jobs (e.g., VMs, MicroVMs, containers) [6]–[8] [40] [22] (see Figure 1a).

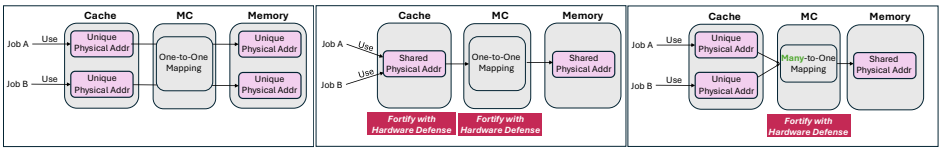
To securely restore memory sharing, prior works [18], [42], [43], [52], [58], [63], [64] propose hardware defenses to mitigate reuse-based side channels in cache and memory. They modify multiple CPU components (see Figure 1b) and modify the OS to utilize the new hardware modifications. To mitigate reuse-based ***cache*** side channels, prior works [42], [43], [52], [64] modify the cache hierarchy —- a complex task due to the multi-level and distributed cache organization today (e.g., a tiled LLC with a separate coherence directory and numerous per-core L1/L2 caches); as a result, these works modify the LLC, coherence directory, L2 and L1 caches, and the memory management unit (MMU), which interfaces with the caches and the OS, so that the OS can use the new cache features. Unlike conflict-based side channel defenses, reuse-based side channel defenses call for OS changes because only the OS is aware of security domains sharing the same physical page.

On the memory side, prior works address reuse-based memory side channels by changing the row policy in the memory controller (MC) to remove row buffer hits [18], [58], [63]. As such, achieving comprehensive protection against reuse-based side channels across both cache and memory would require modifying many hardware components, which is complex and difficult in practice; no hardware defense for memory sharing has ever been incorporated into any real system.

Goal: This paper seeks to create a practical hardware design that would modify the minimum number of components, i.e., just one hardware component since it is hardware defense, and OS - since hardware itself is unaware of security domain, while being capable of comprehensively protecting against reuse-based side channels across both cache and DRAM.

Observation: We observe that a level in the cache/memory hierarchy requires hardware protection only if it uses shared addresses. Thus, ***the fewer levels that use shared addresses, the fewer the hardware components that must be fortified and modified for secure memory sharing.***

Idea: As such, we propose delaying the merging of unique addresses into a shared address until the very end – when the request reaches memory (Figure 1c). We achieve this by enhancing the MC to eliminate the reuse-based side channel in caches, without any modification to caches. We call this idea ***Secure and Practical Endpoint Address Merging*** (or simply ***SEAM***). Under SEAM, the OS allocates different PPNs to different jobs and rely on the MC to merge the different PPNs to the same PPN to share memory. By enabling different jobs to use different PPNs and, thus, different cachelines while



(a) Current system deployment. (b) Prior hardware defenses. (c) SEAM.

Fig. 1: Comparison between existing systems, prior hardware defenses and Secure and Practical Endpoint Address Merging (SEAM).

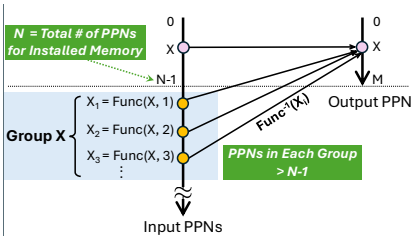


Fig. 2: SEAM uses Table-free merging.

sharing memory,[1] SEAM fundamentally ***eliminates*** reuse-based ***cache*** side channel from memory sharing (see Figure 1c). Compared to prior hardware defenses, SEAM reduces the number of hardware modifications down to just one (see Table I). Also, SEAM has simpler OS modification than prior works because it limits all changes to page allocation and deallocation.

Components to Change for Comprehensively Protecting Against Reuse-based Side Channels	MC	LLC	L2 Caches	L1 Caches	Cache Directory	MMU	OS
When Using SEAM	▲	—	—	—	—	—	▲
When Using Prior Hardware Defenses	▲	▲	▲	▲	▲	▲	▲

TABLE I: Comparison of changes needed to comprehensively protect against reuse side channels in both cache and memory to enable secure memory sharing. Section IV describes in detail the changes needed by prior works.

A practical hardware protection should also incur minimal performance overhead. As such, a key design challenge is how to minimize performance overheads while enhancing the MC to merge physical addresses. While executing memory instructions, existing systems merge virtual addresses to shared physical addresses by relying on the MMU to look up page tables, which record the shared PPNs. Reusing this table-based approach to perform address merging in the MC requires adding a new (i.e., PPN-to-PPN) mapping table to the MC and incurring new latency and bandwidth overheads to perform additional address translation using the new table.

To address the design challenge, we propose a novel table-free merging for the MC. SEAM enhances the MC to merge each fixed group of PPNs to the same PPN (i.e., map all PPNs in Group x to PPN x, see Figure 2); since the group of PPNs that merge to the same PPN is fixed, the output PPN for a given input PPN can be computed directly using a predefined mathematical formula (see $func^{-1}$ in Figure 2), without any table access. To share an arbitrary physical page p ***securely*** across an arbitrary set of jobs, the OS can allocate to the jobs different PPNs among the group of PPNs that merge to p.[2]

This paper makes the following contributions:

- We propose a practical hardware protection to securely restore memory sharing across jobs. It protects against reuse side channels without modifying the caches so that the only hardware that must be modified for full (i.e., cache and memory) protection is the memory controller.
- We evaluate whether SEAM is practical by creating a full-system prototype, consisting of recent Linux kernel and X86 cores. It is ***the very first*** real-system prototype of hardware protection against reuse side channels.
- Our security evaluation of memory sharing across containers shows SEAM successfully protects against reuse-based attacks even in the most ideal attack condition.
- We evaluate performance using containerized workloads. Compared to the security practice deployed in existing systems (i.e., disabling memory sharing across jobs), SEAM slightly improves performance. Compared to an idealization of prior hardware defenses, SEAM is only $< 0.8\%$ slower on average (while being far simpler).

II. Background on Memory and Memory Sharing

This section provides relevant background on the memory hierarchy and memory sharing.

A. Memory Hierarchy

Cache and DRAM are two parts of today's memory system where a shared page or block can reside. This subsection describes the background on cache and DRAM, which is required to understand the security of memory sharing.

1) Cache: A cache is organized as a two-dimensional array of sets and ways, with each element usually representing a 64B cacheline. For example, a 128MB cache with 16-way associativity contains 128MB/(16 * 64B) = 131072 sets. The cache determines which set a block belongs to based on its physical address, which is formed by concatenating the Physical Page Number (PPN) and Page Offset. Specifically, the set index is determined by several bits in the middle of the physical address of a block (see Figure 3). For example, in the 16-way 128MB cache, the set index spans from bit 6 to bit 22. Consequently, blocks from the same physical page typically map to different cache sets, as their set indices will vary based on the address.

2) DRAM: DRAM is organized hierarchically, with the memory controller accessing its modules via a memory channel. Each module can have one or more ranks; each rank contains multiple banks that can operate independently. Each bank contains a two-dimensional array of DRAM cells, organized into rows and columns, along with a row buffer. The address mapper in the MC translates a physical address in a memory

[1] Same as existing system deployment which stores redundant copies of shared blocks.

[2] Although the merging function is fixed, the allocation of actual PPN can still be random because the PPN can be randomly chosen among the fixed group of PPNs, see Section V-B.

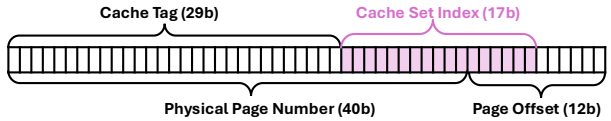


Fig. 3: Layout of the 52-bit physical address of a cacheline in a 128-MB 16-way cache, assuming physical page size is 4 KB. The cache tag (bits 23 and above) identifies which block the cacheline belongs to. Set Index (bits 6 to 22) selects which set in the cache the line maps to. Byte offset (bits 0 to 5) selects the byte within a 64-byte cacheline.

request into a specific DRAM address, including channel, rank, bank, row, and column number (see Figure 4). The MC then encodes these numbers calculated from the physical address in the address field of DRAM commands to access the memory block. When a row is accessed, its entire content is copied into the row buffer. Subsequent read or write operations are then performed from this buffer, rather than directly from the cell array, allowing fast access to data within an open row. As such, the row buffer in DRAM is effectively a cache for the most recently accessed DRAM row for each bank.

Fig. 4: DRAM address mapping. It uses a static 1:1 mapping function.

B. Memory Sharing

The rapidly increasing cost of DRAM, driven by growing memory demands across modern workloads such as AI, makes saving memory increasingly important, elevating memory efficiency to a critical design objective for computing systems.

Historically, systems transparently share identical content across processes, such as when multiple programs load the same binary through a common loader. The goal of memory sharing is to save memory (either code or data) across different programs, unlike *shared memory*, which enables communication across one program's processes and threads.

To support memory sharing, the CPU and kernel introduce the ***read/write*** status bit in PTEs. When two physical pages contain identical content, the OS merges them by mapping both processes' virtual pages to the same physical page and marking the virtual pages as read-only in the PTE; subsequent reads proceed normally, but a write triggers a page fault. The OS resolves the fault via copy-on-write (COW): it allocates a private copy of the page, updates the page table to the new writable mapping, and resumes the faulting process.

There are two kinds of memory sharing.

Allocation-time memory sharing. This type of sharing is established at allocation or process launch time by the OS. Instead of allocating new physical pages, the OS directly maps multiple virtual pages to an existing physical page. This can occur for file-backed code/data segments (e.g., multiple processes sharing the same loaded binary or data file). The OS then protects the merged virtual pages, which can be either code or data file pages, with read-only permission.

Memory deduplication. This type of sharing is established after memory has already been allocated; examples include Linux Kernel Same-page Merging (KSM) and Windows Page Fusion. KSM scans for virtual pages with identical content and merges them into a single physical copy. Like allocation-time memory sharing, KSM protects deduplicated virtual pages with read-only permission to handle future writes with COW.

While the descriptions in this paper primarily focuses on allocation-time memory sharing, the underlying idea is applicable to both types of memory sharing.

III. Threat Model and Side Channels

Below are the threat model and relevant side channels.

A. Threat Model

In the multi-tenant cloud setting considered here, an attacker and a victim program belong to different tenants but may share parts of the data or software stack, as well as hardware resources such as caches/DRAM. The two programs can execute concurrently on different cores or the same core. The attacker is assumed capable of mounting side-channel attacks at any level of the memory hierarchy. An attacker can run malicious software to exploit various side channels to extract secrets from victim workloads co-located on the same CPU.

B. Reuse-based Side Channels

Historically, cloud providers allow memory sharing across different VMs and containers. However, researchers later discover memory sharing opens up a microarchitecture side channel [14], [15], [23], [24], [33], [37], [65]–[67], [69]; the discovery causes cloud providers to turn off memory sharing across different VMs and containers.

Memory sharing opens up the reuse-based side channel, which exploits the reuse of shared code/data. Reuse happens when one container/VM accesses the data that another container/VM recently accessed and suffers from obviously shorter latency (see Figure 5). The attacker can exploit the reuse-based side channel to detect the victim's access pattern for certain shared code/data. If the shared code/data access is indexed by a secret, the attacker can infer the victim's secret.

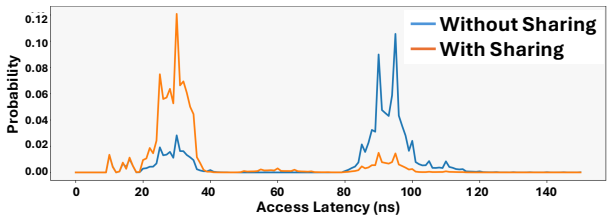


Fig. 5: Probability of access latency for 100 million accesses measured by an attacker program running on a server with an Intel Platinum 8462Y+ CPU. During the measurement window, a victim program running on the same server is accessing the same file (i.e., With Sharing) or a different file (i.e., Without Sharing) as the attacker.

This work focuses on reuse-based cache side-channel attacks that arise from memory sharing across different security domains. Such sharing necessarily requires pages to be mapped as **read-only** in all participating domains; otherwise, writes by one domain would violate the correctness and

isolation of memory values in other domains. As a result, reuse-based cache side channels in our threat model can only occur on read-only shared pages created by memory sharing mechanisms. Writable shared pages across security domains are explicitly disallowed by the OS and are therefore out of scope.

*1) **Reuse-based Side Channel in Cache**:* Flush+Reload [65] is the most common reuse-based cache side channel attack. In this attack, an attacker process first flushes a specific memory block (which is shared with the victim) from the CPU cache (see Figure 6). It then waits for a certain period and reloads the shared code or data to check if it has been accessed by the victim process. If the victim accesses the shared code or data, it will cache the content in the CPU cache, causing quicker access when the attacker reloads it. Victim's access pattern to shared code or data can reveal secrets (e.g., such as cryptographic keys or sensitive data) from private/nonshared pages, since the code/data in private pages can access shared pages. For example, an attacker can infer RSA keys based on which shared addresses the victim accesses.

The victim and the attacker under Flush+Reload can run in either the same core or different cores of the same CPU since they share the same LLC which is used by all cores.

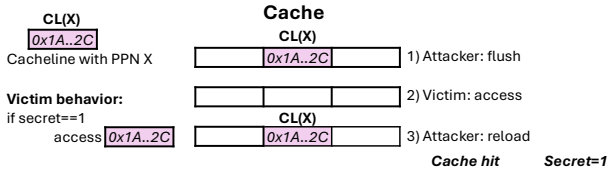


Fig. 6: An example of Flush+Reload attack. The secret of victim has one bit. The victim accesses the shared block when the one-bit secret is 1.

*2) **Reuse-based Side Channel in Memory**:* Memory sharing introduces a reuse-based memory side channel, where an attacker can detect a victim's memory access pattern by exploiting row buffers. Because of the caching behavior of the row buffer, similar to CPU cache, the row cached in the row buffer can also be reused, suffering from a reuse-based side channel. Specifically, when the victim accesses a shared page, the corresponding DRAM row is activated and left open in the row buffer. Subsequent accesses by the attacker to the same shared page can result in a row buffer hit, revealing that the victim recently accessed that row [48].

C. Severity and Impact of Reuse-based Side Channels

Among various side channels, the reuse-based side channel (especially in cache) stands out because it is alias-free, posing a particularly severe security risk. Because the side channel relies on memory shared between the attacker and victim, the attacker can directly observe whether a specific memory block was accessed by the victim just by measuring access latency; this results in a strong and clean side channel, making the attack highly accurate. In comparison, conflict-based cache side channel (e.g., Prime+Probe [45]) suffers from aliasing. Explicitly, conflict-based attacks exploit cache conflicts between processes, making them harder to execute successfully [68] [31] [19] [29] [30] [37] [44] [51] [61] [62]. These attacks suffer from aliasing, revealing only whether any block in a cache set was accessed instead of whether a block with specific address was accessed, which adds noise and reduces precision.

Because reuse-based side channel attacks can leak sensitive information [14], [15], [23], [24], [33], [37], [65]–[67], [69], many cloud providers disable memory sharing across VMs [6], [22], [40]. They even disable memory sharing across VMs of the same user [40] to prevent a successful attack in one VM from spreading to the user's other VMs. Also, VMware disables inter-VM Transparent Page Sharing (TPS) by setting the option $Mem.ShareForceSalting$ by default [11].

D. Security Focus

Our paper focuses on enabling cloud systems to securely restore memory sharing across different jobs (e.g., VMs and containers), instead of memory sharing within jobs. Today's cloud allows memory sharing within a single VM or container [4], which is already considered secure since the sharing occurs only within the same security domain. Similarly, VMware enables intra-VM TPS by default [1].

In this paper, we specifically focus on the reuse-based side channel due to memory sharing. Different from reuse-based side channels, other side channels like conflict-based side channels and branch predictor side channels (e.g., Spectre [33]) exist regardless of memory sharing. Memory deduplication (as opposed to allocation-time memory sharing) also introduces a COW side channel; however, it can be proficiently solved by software-only defenses [35] and, therefore, need not be discussed in this paper.

IV. Related Works

Defenses against Reuse Side Channel in Caches: Prior works mitigate reuse-based cache side channels by altering the cache hierarchy [42], [43], [52], [64], which is complex. In fact, they modify the LLC, coherence directory, L1 and L2 caches, OS, as well as the MMU, which interfaces with the caches and exposes the new hardware features to the OS.

Some prior works [43] [52] [42] [32] [57] attempt to fake cache latencies to eliminate timing differences caused by memory sharing. For example, state-of-the-art work TimeCache [43] fakes cache miss latency when a process first hits on a shared cacheline loaded into the cache by another process. TimeCache modifies all levels of caches [43]. Faking cache miss latency for a cache hit of a shared block in a remote private cache would require modifying the cache coherence directory. TimeCache modifies the OS kernel to store the security bits (which are the metadata used by TimeCache for deciding on access delay) in kernel memory. Exposing security bits from kernel memory to cache would require changing the MMU, since the MMU interfaces with the OS through caches. TimeCache also suffers from costly area overhead in the LLC. For example, assuming a 64-core CPU with two

hardware contexts per core, TimeCache [43] can incur a 31% overhead for each 512b LLC cacheline.[3]

Another state-of-the-art hardware work, ScatterCache [64] modifies the cache hierarchy to use a shared physical address in the cache without sharing cachelines across security domains. Explicitly, ScatterCache applies different keyed hash functions for set indexing of cachelines used by different jobs. Because cacheline reuse can happen at any level of caches, even if different jobs use different cores (e.g., a malicious process in one core can reuse a cacheline in a victim process's private cache in another core if the cacheline holds a shared memory block), ScatterCache requires changes to the whole cache hierarchy, including L1, L2, and LLC. Modifying cache indexing logic under ScatterCache also changes the way of indexing the directory because the directory is also set-associative. ScatterCache also requires modifying the MMU for identifying which security domain a virtual page belongs to and modifying the kernel to select the correct Security Domain Identifier (SDID) for the PTE used when copying data [64].

Defense against Reuse Side Channel in Memory: Addressing the reuse-based side channel in memory requires that accesses to a shared page behave as if the page is not shared. Specifically, MC must eliminate the timing effects caused by row buffer reuse: after one job accesses a shared page, subsequent accesses to the same page by different jobs (before the row timeout) should result in row buffer misses, not row buffer hits. Prior defenses [18] [58] [63] propose removing all row buffer hits to avoid this reuse.

Summary: Leveraging prior hardware defenses to comprehensively protect against reuse-based side channels across the memory hierarchy (i.e., in both cache and memory) would require modifying many hardware components including the MC, LLC, L1/L2 caches, cache directory, and MMU (see summary in Table I). Changing so many components is intrusive and difficult to realize in practice.

V. Endpoint Address Merging

In this paper, we aim to design a practical hardware protection against reuse-based side channels that modifies the minimum number of hardware components – ideally just one. Specifically, we aim to eliminate the reuse-based cache side channel without modifying the cache hierarchy so that the only hardware that must be modified is the MC.

We observe that a cache or memory level requires hardware protection only if it uses shared addresses. Thus, reducing the levels that use shared addresses can directly minimize the number of hardware components to modify for secure sharing.

Based on this observation, we propose delaying merging unique addresses into a shared address until the end – when the request reaches memory. This reduces the total number of hardware components to modify down to just one – the MC. We call this idea Secure and Practical Endpoint Address Merging (*SEAM*).

[3]Each cacheline needs a 32-bit Tc to record when it is inserted into cache and 1-bit s-bit per hardware context, so the overhead for supporting 128 hardware contexts is $(1b * 128 + 32b)/512b = 31\%$ in total.

Figure 7 shows the architecture overview of SEAM with an example of two jobs sharing the same page. The only hardware change is adding a PPN merging unit into the MC. When the OS allocates different PPNs (X_1 and X_2) to different jobs (J_1 and J_2), SEAM merges the different PPNs to the same PPN to share memory. Because SEAM enables different jobs to use different PPNs and, thus, different cachelines while sharing memory, SEAM eliminates all reuse-based side channels from ***all caches***. As we will discuss in Section V-B, SEAM can also help to more efficiently protect against reuse-based side channels in memory by enabling the MC to identify whether the current memory accesses are coming from different jobs or the same job.

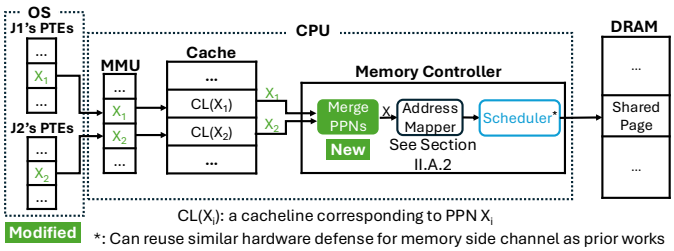


Fig. 7: Architecture overview of SEAM.

Compared to conventional memory sharing, SEAM is secure against reuse-based side channels and loses the benefit of cache saving (see Table II).

Sharing Mechanisms	Security	DRAM Saving	Cache Saving	Fragmentation
Conventional Memory Sharing	✗	✓	✓	✓
SEAM	✓	✓	✗	✓

TABLE II: Comparison between SEAM and conventional memory sharing. For both SEAM and conventional memory sharing, deduplication and copy-on-write break physical contiguity, scattering pages across memory and increasing external fragmentation over time.

Design Challenge: The key design challenge is that enhancing the MC to merge PPNs can introduce new address translation overheads. A practical hardware protection should incur little or no performance overhead. While executing memory instructions, existing systems merge virtual addresses to shared physical addresses by relying on the MMU to look up page tables, which record the shared PPNs. Adopting this table-based approach to perform address merging in the MC would require the MC to access a new PPN-to-PPN mapping table (see Figure 8). Address translation is already a memory performance bottleneck for real-world data center workloads, as exemplified by the extensive research on address translation [13], [39], [46], [56]. Requiring the MC to access an added table can further slow down memory accesses, as data accesses can only start after accessing the table to determine where to access. Also, accessing the table entries incurs a bandwidth overhead. Adding a translation cache to speed up the table access further incurs an SRAM area and latency overhead (i.e., adding cache hit latency per access).

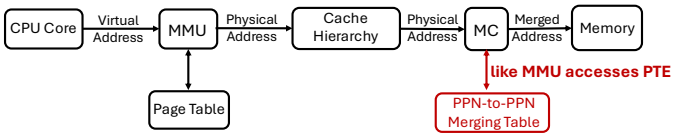


Fig. 8: Design Challenge: Reusing table-based approach for merging addresses in the MC would harm performance.

A. Table-free Address Merging

To address the above design challenge, SEAM enhances the MC with a novel table-free merging logic that merges each fixed group of PPNs to the same PPN (i.e., map all PPNs in Group X to PPN X, see Figure 2). Given an input PPN, the merging unit directly computes the output PPN using a predefined mathematical formula, without any table access. The OS can utilize the table-free merging hardware to share memory securely across different jobs by allocating to the jobs different PPNs from the same group.

Composition of Each Merge Group: If not done carefully, merging multiple PPNs to the same PPN can leave some physical locations unused when merging the original PPN for accessing that memory location with another PPN, wasting memory (see Figure 9.a). To avoid this, we compose each fixed merge group strictly from PPNs beyond the installed memory range so that PPNs in the installed memory range always point to the same memory location as existing systems, without wasting memory (see Figure 9.b) (Despite of the fixed merging function, the allocation of actual PPN can still be random because the PPN can be randomly chosen among the fixed group of PPNs, see Section V-B). Since the PPNs beyond the installed memory range can only be used for sharing, but not standalone, we refer to them as ***sharing-only PPNs***. We refer to how many sharing-only PPNs can map to the same conventional PPN as the *merge group size*.

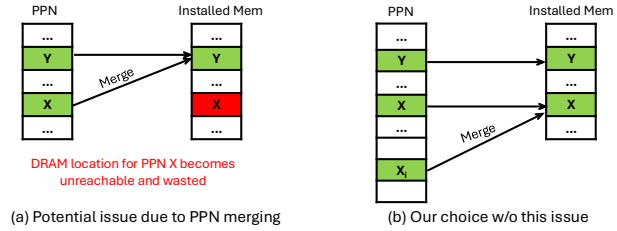


Fig. 9: Different choices of PPNs to merge; x_i refers to a sharing only PPN.

The composition of each group of sharing-only PPNs can also impact cache hit rate. Ideally, each group should consist of PPNs that use different cache sets. Otherwise (e.g., if the PPNs in a group use the same cache sets), different jobs sharing memory by using different PPNs from the same group will cache their identical copies of memory blocks into the same sets, increasing set contention and harming cache hit rate.

Based on the considerations above, let the merge group size be S, SEAM calculates the i-th sharing-only PPN in a group as:

$$X_i = N + X \times S + i. \quad (1)$$

where N is the total number of conventional PPNs, X is the conventional PPN in the range $[0, N)$. The i-th sharing-only PPN in the group is said to have the ***group member ID*** of i. Because i ranges over all integers in $[0, S)$, and each value of i generates a distinct sharing-only PPN within the same group, these PPNs necessarily differ in their lowest $\log_2 S$ bits (see Figure 10). Since the lowest bits determine the cache set index (Figure 3), cachelines with different sharing-only PPNs naturally map to different sets without requiring any changes to the cache logic.

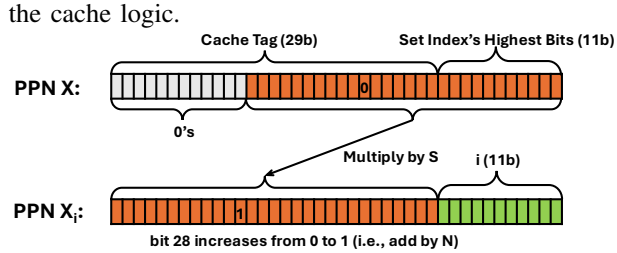


Fig. 10: An example of calculating a 40-bit sharing-only PPN for a system with 1TB memory. Bits are ordered from most significant (left) to least significant (right). Here $S = 2^{11}$, $N = 2^{28}$ and i has 11 bits.

Because the maximum value of the sharing-only PPN is $N + N \times S - 1$, to ensure all sharing-only addresses fit within limited number of PPNs, we require $N + N \times S - 1 \leq T - 1$ (i.e., $S < T/N$), where T is the total possible number of PPNs. As an example, assuming a 4 KB page size and the 4PB physical address space being used today (see Section 5.5 in Intel Architectures Software Developer's Manual [25]), $T = 2^{52}/4\,\text{K} = 2^{40}$. For a 1 TB memory system, $N = 1TB/4\,\text{KB} = 2^{28}$; as such, $S < T/N = 2^{40}/2^{28} = 2^{12}$. (In other words, the total number of sharing-only PPNs in a group is at most $2^{12} - 1$.)

Merging Microarchitecture: SEAM's table-free PPN merging unit in the MC inverses the calculation of the sharing-only PPN. The inverse calculation to obtain the original conventional PPN from a sharing-only PPN is:

$$RegularPPN = (SharingOnlyPPN - N)/S \quad (2)$$

where '/' is integer division.

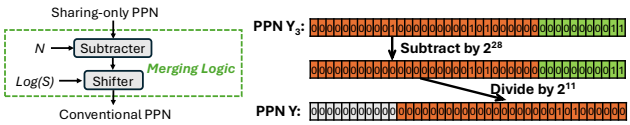
To minimize the hardware overhead of division logic, we choose a group size S that is a power of 2 so that the division reduces to right shifting. For example, for a 1TB memory system, one can choose $S = 2^{11}$, which is the biggest power of 2 smaller than 2^{12}. The subtraction here is fast because the big value of N restricts the carry propagation, and only a small number of higher-order bits of $sharingOnlyPPN$ and N participate in the subtraction.

Under the common cases, the value of S (i.e., 2^{11}) is big enough to support memory sharing since a machine rarely runs 4000 jobs. Under the rare case that more than S jobs have the identical page content, the OS can duplicate the physical page for each additional group of S jobs and share the new copy among the new group.

The hardware logic for table-free merging contains one subtractor and shifter (see Figure 11a) to reverse the calculation in Figure 10. Figure 11b shows an example of the PPN merging.

B. Security Analysis

1) Security in Cache: SEAM ensures the same security as turning off memory sharing. The identical security comes



(a) Merging logic. (b) An example of merging PPN.

Fig. 11: A) PPN merging circuit and B) an example of the merging calculation. The example assumes the same system as Figure 10.

from two identical behaviors: the cacheline reuse behavior and cache conflict behavior.

Reuse Behavior: When turning off memory sharing, the OS allocates different PPNs to different jobs to prevent cacheline reuse across jobs. SEAM preserves this property: OS assigns different jobs distinct PPNs, preventing cacheline reuse. Below, we prove that Equation 1 ensures sharing-only PPNs assigned to different live jobs are never equal regardless of whether they belong to the same group.

Suppose job A has a sharing-only PPN X_i. If job B has a sharing-only PPN X_j in the same group:

$$X_i - X_j = (N + X \times S + i) - (N + X \times S + j) = i - j.$$

Because $i \neq j$, $X_i \neq X_j$.

Now, suppose job B has a sharing-only PPN Y_j in another group (i could be either equal or not equal to j):

$$X_i - Y_j = (N + X \times S + i) - (N + Y \times S + j) = (X - Y) \times S + (i - j).$$

Because $X \neq Y$, we know $|X - Y| \geq 1$ and thus $|(X - Y) \times S| \geq S$. Also, because $0 \leq i, j < S$, $|i - j| < S$. As such, $X_i - Y_j = (X - Y) \times S + (i - j) \neq 0$; therefore, $X_i \neq Y_j$.

Conflict Behavior: As mentioned in Section III-D, in this paper, we focus on reuse-based side channels and do not consider conflict-based cache side-channel attacks, as they exist even when memory sharing is turned off. Ideally, SEAM should not make the conflict-based side channel attack easier compared to turning off memory sharing. Actually, SEAM preserves the same cache set conflict behavior as turning off memory sharing; under systems that turn off memory sharing, pages from different jobs but with the same content are mapped to unrelated cache sets.

Under SEAM, to achieve the same conflict behavior as turning off sharing, the OS randomly selects the group member ID from the pool of unused IDs to generate the sharing-only PPN, ensuring that different jobs' copies of a shared page are placed in unrelated cache sets. For example, in a 16-way 128 MB LLC, there are in total $128MB/(4KB \times 16) = 2048$ possible collections of cache sets for storing one 4 KB physical page, where each collection contains $4KB/64B = 64$ cache sets. In systems turning off memory sharing, a page can be mapped to any of these 2048 collections of cache sets. If SEAM uses 2048 as the group size for calculating sharing-only PPNs, each copy of a shared page can be mapped to any of the 2048 collections of cache sets.

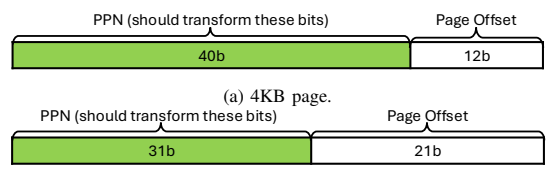
2) Security in Memory: As mentioned in Section IV, prior defenses against memory side channel [18] [58] [63] remove all row buffer hits to avoid the row buffer reuse due to memory sharing. However, this row policy harms memory performance due to eliminating the benefit of row buffer hits.

To avoid the overhead of a closed-row policy, we note the group member ID in sharing-only PPNs can help the MC distinguish accesses from different jobs. The MC can use sharing-only PPNs to determine whether closing the row in the row buffer and reopening it is necessary for security. Specifically, after accessing a shared page, if subsequent requests target the same DRAM row and have the same sharing-only PPN, closing the row buffer is unnecessary because reusing the row buffer is secure within the same job. To support this check, the MC can record for each bank the sharing-only PPN of the most recent request.

C. Generality

Huge Pages: In this paper, we focus on 4KB pages, as current systems typically only support sharing at the 4KB granularity and do not share 2MB huge pages. However, SEAM can be extended to huge pages with minimal changes.

When merging PPNs for huge pages, the MC should merge only the most significant bits of the requested address, since huge-page PPNs contain fewer bits than those of 4KB pages (see Figure 12). As such, the MC must first determine the page size of each memory request; one way to do this is to use different address ranges for 4KB and huge pages, so that the page size of a request can be conveyed simply through the request's address range.



(a) 4KB page.

(b) 2MB page.

Fig. 12: The physical address bits to transform for 4KB pages and 2MB pages.

We extend Equation 1 as:

$$X_i = EN + X \times S + i. \tag{3}$$

where $EN = N$ for 4KB pages while $EN = (N + N \times S)/(2MB/4KB)$ for huge pages. For example, for a server with 1TB memory, we let $S = 2^{10}$. When the physical page is a 4KB ($2^{12}B$) page, $EN = N = 2^{28}$. When the page is a 2MB ($2^{21}B$) page, $EN = (N + N \times S)/(2^{21}/2^{12}) = 1025 \times 2^{19}$. In this way, the sharing-only addresses of 4KB pages locate at $[1T, 1025T)$ while sharing-only addresses of huge pages always locate at $[1025T, 2049T)$. Correspondingly, we extend the Equation 2 for calculating conventional PPN as:

$$RegularPPN = (SharingOnlyPPN - EN)/S. \tag{4}$$

Multiple MCs: Modern CPUs may interleave the memory blocks of a page across MCs. For example, the CPU may route the even blocks within a page to MC0 and route the odd blocks within a page to MC1. Because SEAM preserves the page offset for every block, SEAM requires no changes to the interleaving pattern. As such, each MC merges the sharing-only PPNs only for the memory requests it receives (e.g., MC0

merges the PPN X_i to X for requests to even-indexed blocks with PPN X_i while MC1 merges the PPN X_i to X for requests to even-indexed blocks with PPN X_i.)

Memory Encryption: Existing secure memory systems with memory encryption [26] [9] use some PPN bits to encode extra information. AMD SEV [9] uses the most significant bits (MSB) as C-bit to tag the encrypted pages. Similarly, Intel MKTME [26] encodes key identifiers using several MSBs in the PPN. To ensure SEAM's compatibility with SEV, one approach is to configure the MC to skip PPN merging when the C-bit is set. For MKTME, one approach is to reserve one of the MSBs as an encryption indicator bit and also configure the MC to skip PPN merging when this bit is set. This excludes encrypted pages from merging operations, preserving system correctness.

D. Discussion on Depletion of Sharing-only PPNs

Since OS allocates a unique sharing-only PPN for each job sharing the same page, an attacker might attempt to launch multiple jobs that share the same page, thereby consuming available sharing-only PPNs in the same group and, thus, reducing the randomness of cache set conflicts for later victim jobs. The decrease of randomness in cache conflict may expose the system to conflict-based side channel.

However, the OS only needs to allocate a new sharing-only PPN when sharing a page used by one security domain (e.g., a job, container, VMs etc) with another security domain. When sharing a page across processes within the same security domain, the OS can reuse the same PPN instead of allocating new sharing-only PPNs since sharing within the same security domain is already safe in current systems (see Section III-D). As such, this attack is impractical: mounting it would require launching hundreds to thousands of jobs (and thus substantial CPU, memory, storage, and network resources), which quickly exceeds the limited capacity of a single server.

VI. Methodology

We prototype SEAM in a real system. As shown in Figure 13, the server includes an FPGA board that implements Compute Express Link (CXL) [2] memory controller and contains 16 GB of on-board. The server is also equipped with the 32-core Intel EMR Gold 6530 CPU (see Figure 13). The rest of this section describes SEAM's modification on both FPGA hardware and OS.

Fig. 13: Real-system prototype of SEAM.

A. Implementation of SEAM

Since the integrated memory controllers (IMCs) for accessing local memory are unmodifiable, we implement SEAM within the FPGA-based CXL memory controller (see Figure 14). To clarify, SEAM is orthogonal to CXL memory, and general for all kinds of memory; we just use the FPGA-based CXL memory to evaluate and verify SEAM. As such, our evaluation just accepts CXL memory's higher latency over host memory [38] without any CXL-related optimizations like migrating hot pages to local memory and cold pages to CXL memory; we use numactl to bind the entire application memory to the CXL memory.

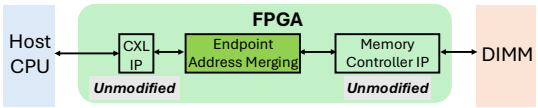


Fig. 14: Overview of the real-system implementation of SEAM.

To support sharing-only PPNs which exceed the range of installed memory, we also configure the FPGA to expose 512 GB to the OS, which extends the number of PPNs of the 16GB CXL memory by 32X. Explicitly, we reserve the upper 496 GB (addresses 16-512 GB) for sharing-only PPNs via the Linux memmap GRUB boot option [49] so that only the lower 16 GB is visible to user space.

Since our goal is to design a practical hardware protection that saves on all modifications to cache-related hardware components, we focus our implementation and evaluation on demonstrating SEAM can securely eliminate the reuse-based side channel ***in the cache***. Thus, in terms of DRAM side channel, we did not implement prior hardware defense (e.g., [18] [41]) against DRAM side channel; also, we lacked access to the internals of the memory controller IP on the FPGA.

B. How to Use SEAM

In data centers today, different jobs are typically placed in different VMs, microVMs, or containers to ensure isolation and manageable resource sharing. Under Linux, each VM [54], microVM [7], or container [53] generally runs in its own control group (cgroup) [55], which allows the host to manage hardware resources such as CPU time, system memory, network bandwidth for each job. To be generally applicable across VMs, microVMs, and containers, the OS portion of our prototype specifically targets cgroups by allocating different sharing-only PPNs when sharing a physical page *across different cgroups*.

To deploy SEAM in a real system, the OS must ensure cache coherence when pages transition between non-shared and shared states and when shared pages are later deallocated. When a physical page is first transitioned from non-shared to shared, the OS issues `CLWB` to write back all cache lines belonging to that page, ensuring that memory contains the most up-to-date values for subsequent accesses. Because pages shared across security domains are read-only, this writeback is required only once at the point of transition. When a shared physical page is deallocated, the OS flushes all associated cache lines using `CLFLUSH` to remove any stale data remaining in the cache. Unlike `CLWB`, `CLFLUSH` evicts cache lines after writing back dirty data. Consequently, later accesses to the page miss in the cache and retrieve correct data from memory. Figure 15 shows an example of how SEAM is used.

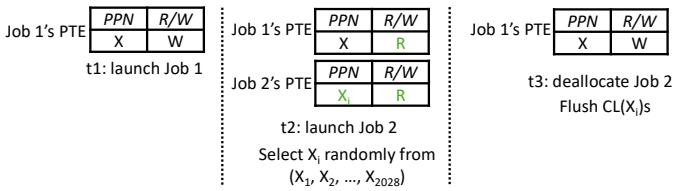


Fig. 15: An end-to-end example of using SEAM, where two jobs share the same physical memory. Here, X is a conventional PPN, while X_i is a sharing-only PPN that merges to X.

The cache writeback and flushing added to page allocation do not bring any new side channel in cache because they keep cache hits and misses unchanged compared to systems without memory sharing. First, the writeback keeps the $CL(X)s$ in cache and, thus, has no effect on the cache hits and misses of the first job. Second, the flushing only invalidates cachelines for terminated jobs and, thus, has no effect on cache hits and misses of running jobs.

We modify Linux kernel 5.10.235 to use SEAM by adding 500 lines of code. Compared to prior approaches, SEAM requires only minimal OS modifications, confined to page allocation and deallocation paths, where standard cache maintenance instructions are invoked during page state transitions.

SEAM assumes a correct and trusted OS kernel, consistent with prior work on OS-managed cache and memory isolation mechanisms. The correctness of cache maintenance operations (e.g., `CLWB`/`CLFLUSH`) therefore relies on the OS correctly enforcing page state transitions. If the OS kernel is compromised or malicious, an attacker can directly access or modify another tenant's memory through OS-level privileges, without requiring cache side-channel attacks. Such threats are orthogonal to SEAM and are typically addressed using multi-tenant memory encryption mechanisms (e.g., Intel MKTME or AMD SEV-SNP), which are outside the scope of this work.

C. Overheads

Hardware Logic Overhead. The hardware logic for SEAM requires only a few bitwise operations (see Figure 11a). ASIC synthesis using Synopsys [60] and the 7nm tech node reports 0.2 ns latency and 0.0000667 mm^2 area. On FPGA synthesis, SEAM fits within the existing timing slack of the default 300Mhz clock rate without needing a new pipeline stage.

OS Memory Overhead. The memory overhead stems from two components. First, each page object adds an 8-byte pointer to use_tracker, accounting for only 0.2% overhead. Second, the per-page, dynamically resized use_tracker array. Each element is compacted into 8B: a 2-byte cgroup_id, 2 byte for group_member_id, and 4B for map_count. The resulting overhead scales with the number of co-located containers and the fraction of shared pages. In a representative deployment with 40 containers [17], assuming all pages are mapped and half of them are shared by all containers, the use_tracker overhead is around 4%, while saving 50% of memory (as 50% of pages are shared 40 times each in the example). Figure 16 shows the effective memory saving for different number of containers under SEAM, which excludes the memory overhead from original memory saving due to sharing.

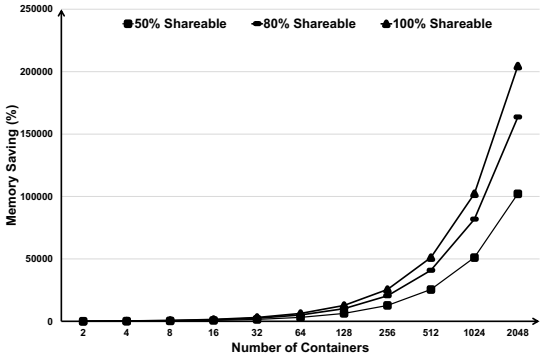


Fig. 16: Effective memory saving under SEAM. Here we assume 50%, 80%, and 100% of the memory trace is shareable separately. The memory saving is normalized to the system memory size.

VII. Evaluation

A. Security Evaluation

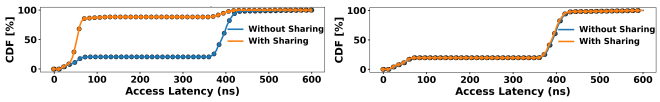
Security Against Flush+Reload: Since the benefit of SEAM over prior hardware defense is removing all cache-related modifications, we focus our security evaluation on demonstrating SEAM can securely protect against reuse-based side channel ***in the cache*** despite no modifications to caches. We test SEAM using a publicly released Flush+Reload attack code [3]. Both the attacker code and the victim code mmap the same file to their own array. The file contains 2048 128-byte elements. The attacker code first flushes the shared array, then waits for the victim to use a secret-dependent index to access an element. Afterward, the attacker reloads each element in the array and measures access latency. The element with the lowest latency reveals which entry was accessed, allowing the attacker to infer the victim's secret.

We run the attacker and victim on the same CPU core to evaluate the most ideal attack condition; if SEAM can securely protect against this most ideal attack scenario, it can protect against less ideal attack scenarios.

Our evaluation on Flush+Reload shows, without SEAM, the attack consistently succeeds—the secret value recovered by the attacker perfectly matches the actual secret. In contrast, with SEAM enabled, the attack consistently fails, as the attacker recovers incorrect values. We repeat the attack with and without SEAM separately for 1 million times and observe the success rate of getting the correct key is around 0.05%, matching with the success ratio of random guessing, which is $1/2048 = 0.05\%$.

Access Latency Distribution: Accesses to shared memory blocks can be faster than accesses to non-shared blocks, which forms the basis of reuse-based side channels. We evaluate the impact of SEAM on this access latency distribution by comparing memory latency across when letting the attacker and victim programs access the same file or access two identical copies of the same file. To increase the diversity of attack scenarios that we evaluate, this experiment runs the victim and attackers on different cores so they only share the

LLC. The LLC capacity is large enough to hold all cache blocks of one file, yet insufficient to hold the blocks of two different files.



(a) Without SEAM. (b) With SEAM.

Fig. 17: Cumulative distribution of access latencies with and without memory sharing, measured both with and without SEAM enabled.

Figure 17 presents the latency distribution of 100 million accesses with and without SEAM enabled, on the server where it is implemented. In Figure 17a, two distinct peaks clearly separate the latencies for with and without memory sharing, revealing the presence of a reuse-based side channel in systems without secure sharing. In contrast, Figure 17b shows that with SEAM enabled, the latency distributions for with and without sharing become indistinguishable, as both no longer use shared addresses in caches. We also conduct a Kolmogorov-Smirnov test on 100,000 access events to compare the timing distributions of accesses with and without sharing under SEAM. The resulting high p-value of 0.83 indicates no statistically significant difference between the two distributions. This concludes the timing behavior of memory accesses are indistinguishable between with and without sharing.

B. Performance Evaluation

We evaluate SEAM in a realistic multi-tenant environment where multiple ***container instances*** colocate on the same server and share read-only code libraries, data files, and other common resources. Cross-VM sharing, however, requires memory deduplication, which needs more software fixes to be secure (see Section III-D). To keep our claims scoped, we present container performance evaluations only. Although our experiments use containers, SEAM is also applicable to VMs.

Benchmarks: We evaluate multiple workloads including DNN-based image classification [34], TPC-H queries [50] to SQLite database [21], OpenSSL [5], SPEC CPU 2017 [12] and PARSEC [10]. For TPC-H, we run four workloads separately, with each workload executing five or six different queries, covering the diversity of query patterns.

Comparison against Existing Systems: We compare the performance of SEAM with our first baseline: today's data center systems, which disable memory sharing across cgroups in fear of reuse-based side channel. We run two, four, and eight containers with each benchmark's one instance in separate containers. Because each container also uses some private/non-shareable memory, the memory size of which increases with more containers, we can run only up to eight containers within the 16 GB of CXL memory. We measure the average running time of each benchmark in each container across three times and compute the normalized performance of each benchmark under SEAM by comparing its average runtime to the baseline.

SEAM improves performance by 3%, on average, across different numbers of containers, compared to systems turning off memory sharing (see Figure 18). The baseline of disabling memory sharing is slower due to requiring the OS to load many identical pages into duplicate memory locations.

For a few benchmarks, SEAM is $< 1\%$ slower. The degradation comes from the software overhead of cache writeback and flushing when allocating shared pages (see Section **??**) (note that these are not needed when allocating unique pages). Our evaluation shows that SEAM increases the allocation time of shared pages by 0.3 µs per page; this is small when amortized across program lifetime.

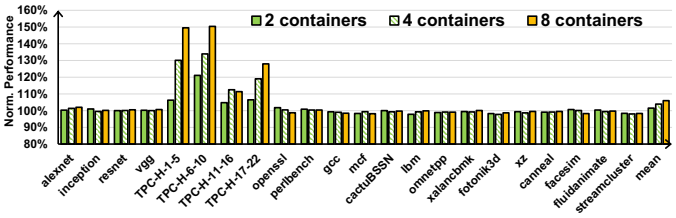


Fig. 18: The performance of SEAM normalized to turning off memory sharing.

While SEAM has similar performance as the baseline of disabling memory sharing, ***the key benefit of SEAM is restoring memory sharing.*** Figure 19 shows the total number of sharing-only PPNs allocated to different workloads. The figure only shows real (e.g., DNN and SQLite) workloads; the number of sharing-only PPNs for benchmark workloads (e.g., SPEC and Parsec) would be harder to see due to being orders of magnitude lower. The number of sharing-only PPNs increases proportionally with the number of containers. Each sharing-only PPN allocated saves 4KB of memory.

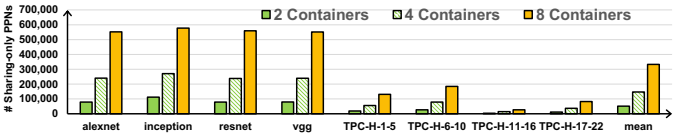


Fig. 19: The maximum number of sharing-only PPNs allocated for DNN and SQLite under SEAM with different numbers of containers.

Under eight containers, the corresponding memory saving is 1.3 GB, on average across the workloads. For reference, without memory sharing, the total memory footprint across eight containers is on average 4.8 GB across the evaluated workloads. The four DNN workloads share the same PyTorch library code via the page cache, leading to a similar amount of memory sharing across workloads. Models (weights) for DNN remain non-shared because Python file loader (i.e., torch.load()) loads the weight file into process-private heap memory rather than memory-mapping the file with mmap().

Comparing Performance against Prior Hardware Defenses: In addition to comparing to existing systems, we also compare SEAM against prior works on hardware defenses. Compared to this second baseline, SEAM has the same behavior of PTE misses since SEAM keeps TLB and caching of page table entries unchanged as page tables are unique to each process. However, SEAM eliminates use of shared addresses in caches and, thus, caches multiple copies of the same memory block for different jobs, potentially increasing cache misses.

We emulate the second baseline by measuring workload performance under memory sharing; this presents an idealization of the prior hardware defense free of all software and hardware performance overheads. We use Intel rdtset [27] to limit the cache size to one way per two containers.[4] As shown in Figure 20, SEAM leads to only a modest performance degradation—on average 0.8% even under eight containers — compared to the baseline system with memory sharing. The worst case slowdown (i.e., TPC-H-17-22) is 4.5%.

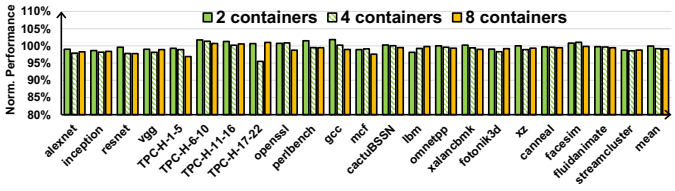


Fig. 20: Performance of SEAM normalized to emulation of prior hardware defense for sharing (i.e., turning on memory sharing without any hardware or software overheads assuming the defenses are free). Some benchmarks show 1% higher performance due to inherent variation in real system measurements.

We also emulate the effect of SEAM on LLC MPKI (Misses Per Kilo Instructions). As Figure 21 shows, under SEAM, most benchmarks' LLC MPKI increases by < 0.2 compared to turning on memory sharing. The increase is less than 0.1 on average even when running eight containers. The increase is small for most workloads because shared blocks (e.g., shared libraries) are typically frequently accessed and remain in the L1 or L2 caches, resulting in few LLC misses. This small increase on LLC MPKI explains the modest peformance degradation shown in Figure 20.

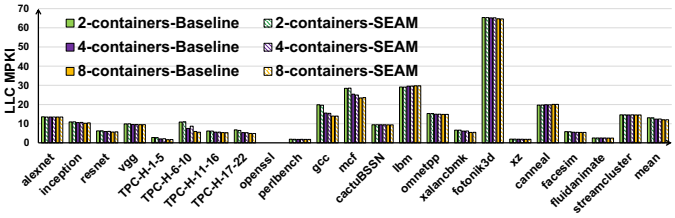


Fig. 21: The MPKI of SEAM and the emulation of prior hardware defense turning on sharing.

VIII. Other Related Works

Prior works [16], [47], [59] have proposed hardware deduplication to enhance the MC to transparently deduplicate memory in hardware; the goal is to deduplicate more than the OS, unlike our goal of designing a ***practical*** hardware protection.

The difference in goals leads to vastly different designs. To achieve the minimum hardware modification needed to protect against reuse side channel, SEAM's entire ***entire*** hardware modification is adding a hardware-friendly stateless merging logic to the MC, retaining the role of identifying and deciding what to share ***with the OS***. To deduplicate more values, hardware memory deduplication identifies and decides ***in hardware*** which blocks, instead of pages, to deduplicate. This requires adding complex hardware logic to manage large data structures that consume large (e.g., 8% [16] to 12.5% [47]) fraction of DRAM, significantly impacting performance (e.g., needing fine-grained block-level translation per access). Furthermore, the hardware-managed data structures are statically-sized (including the various exception regions [16], [47]) and are, thus, vulnerable to overflow and require complex, undescribed OS support for correctness. The static structures also cap the deduplication ratio at 16X [59], unlike SEAM (i.e., can scale to $> 1000X$). Lastly, hardware deduplication also only specifically targets deduplication, unlike SEAM, which is applicable for both allocation-time sharing and deduplication.

Some prior works [36] [20] use Intel's cache allocation technology (CAT) [28] to address side channels in the LLC. By inserting cachelines of different processes into independent cache ways, CAT prevents cache eviction between different users. However, if a memory block is shared, CAT inserts only one copy of the block in the LLC. Enhancing CAT to insert duplicate copies of the same physical memory block would require complex changes to the cache coherence protocol. Even after the complex change, the maximum number of ***non-overlapping*** cache partitions is limited by the LLC associativity. For example, a 20-way LLC can support at most 20 non-overlapping partitions, with each partition occupying at least one cache way, limiting secure isolation only for at most 20 concurrent jobs.

IX. Conclusion

Prior works on hardware defenses against reuse-based side channels require modifying multiple hardware components throughout the CPU. To design a practical hardware protection that modifies a minimum number of hardware components, we observe that only hardware using shared addresses needs protection; thus, reducing the number of components that use shared addresses can minimize the required hardware changes.

As such, we propose Secure Endpoint Address Merging (SEAM) to delay merging into shared addresses until the very end – when the request reaches memory; SEAM reduces the total number of hardware components to modify for protecting against both cache and memory reuse-based side channels down to just one - the memory controller.

We implement a real-system prototype of SEAM to verify whether it is indeed practical. The prototype is the *very first* real-system prototype of hardware protection against reuse side channel. Our real-system evaluations show it securely protects against reuse attacks while sharing memory and matching the performance of disabling memory sharing across jobs (i.e., the current practice in data centers out of security concerns).

References

[1] "Additional transparent page sharing management capabilities and new default settings," [Online; accessed Sep. 2025]. [Online]. Available: https://knowledge.broadcom.com/external/article?legacyId=2097593

[2] "Compute express link." [Online]. Available: https://www.computeexpresslink.org

[4] Because the CPU has 32 cores and 15-way LLC, assuming each core is running one container, two containers share $(2/32) * 15 = 1$ LLC way.

[3] "Hands-on hardware side channels lab," [Online; accessed Aug. 2025]. [Online]. Available: https://github.com/MATCHA-MIT/SHD-CacheAttackLab.git

[4] "How amazon ecs manages cpu and memory resources," [Online; accessed Sep. 2025]. [Online]. Available: https://aws.amazon.com/blogs/containers/how-amazon-ecs-manages-cpu-and-memory-resources

[5] "Openssl library," https://www.openssl.org/, 2025, accessed: 2025-Nov.

[6] Amazon, "The ec2 approach to preventing side-channels." [Online]. Available: https://docs.aws.amazon.com/whitepapers/latest/security-design-of-aws-nitro-system/the-ec2-approach-to-preventing-side-channels.html?

[7] Amazon, "Logical separation on aws: Aws whitepaper," July 2020. [Online]. Available: https://docs.aws.amazon.com/pdfs/whitepapers/latest/logical-separation/logical-separation.pdf#serverless-and-containers

[8] Amazon, "Security overview of aws lambda," December 2022. [Online]. Available: https://docs.aws.amazon.com/pdfs/whitepapers/latest/security-overview-aws-lambda/security-overview-aws-lambda.pdf

[9] AMD, "Strengthening vm isolation with integrity protection and more," 2020. [Online]. Available: https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/SEV-SNP-strengthening-vm-isolation-with-integrity-protection-and-more.pdf

[10] C. Bienia, S. Kumar, J. P. Singh, and K. Li, "The parsec benchmark suite: Characterization and architectural implications," in *Proceedings of the 17th international conference on Parallel architectures and compilation techniques*, 2008, pp. 72–81.

[11] BROADCOM, "Security considerations and disallowing inter-virtual machine transparent page sharing," [Online; accessed Sep. 2025]. [Online]. Available: https://knowledge.broadcom.com/external/article/318959/security-considerations-and-disallowing.html

[12] J. Bucek, K.-D. Lange, and J. v. Kistowski, "Spec cpu2017: Next-generation compute benchmark," in *Companion of the 2018 ACM/SPEC International Conference on Performance Engineering*, 2018, pp. 41–42.

[13] S. Chai, J. Zhang, J. Kim, A. Wang, F. Chung, J. Stojkovic, W. Jia, D. Skarlatos, J. Torrellas, and T. Xu, "{EMT}: An {OS} framework for new memory translation architectures," in *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*, 2025, pp. 711–729.

[14] G. Chen, S. Chen, Y. Xiao, Y. Zhang, Z. Lin, and T. H. Lai, "Sgxpectre: Stealing intel secrets from sgx enclaves via speculative execution," in *2019 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2019, pp. 142–157.

[15] Y. Chen, L. Pei, and T. E. Carlson, "Afterimage: Leaking control flow data and tracking load operations via the hardware prefetcher," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 16–32.

[16] D. Cheriton, A. Firoozshahian, A. Solomatnikov, J. P. Stevenson, and O. Azizi, "Hicamp: architectural support for efficient concurrency-safe shared structured data access," *ACM SIGPLAN Notices*, vol. 47, no. 4, pp. 287–300, 2012.

[17] Datadog, "8 surprising facts about real docker adoption," 2018. [Online]. Available: https://www.datadoghq.com/docker-adoption/?utm_source=chatgpt.com

[18] P. W. Deutsch, Y. Yang, T. Bourgeat, J. Drean, J. S. Emer, and M. Yan, "Dagguise: mitigating memory timing side channels," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 329–343.

[19] C. Disselkoen, D. Kohlbrenner, L. Porter, and D. Tullsen, "{Prime+Abort}: A {Timer-Free}{High-Precision} l3 cache attack using intel {TSX}," in *26th USENIX Security Symposium (USENIX Security 17)*, 2017, pp. 51–67.

[20] X. Dong, Z. Shen, J. Criswell, A. L. Cox, and S. Dwarkadas, "Shielding software from privileged {Side-Channel} attacks," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 1441–1458.

[21] K. P. Gaffney, M. Prammer, L. Brasfield, D. R. Hipp, D. Kennedy, and J. M. Patel, "Sqlite: past, present, and future," *Proceedings of the VLDB Endowment*, vol. 15, no. 12, 2022.

[22] Google, "7 ways we harden our kvm hypervisor at google cloud: security in plaintext." [Online]. Available: https://cloud.google.com/blog/products/gcp/7-ways-we-harden-our-kvm-hypervisor-at-google-cloud-security-in-plaintext?m=1

[23] D. Gruss, C. Maurice, K. Wagner, and S. Mangard, "Flush+ flush: A fast and stealthy cache attack," in *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*. Springer, 2016, pp. 279–299.

[24] D. Gullasch, E. Bangerter, and S. Krenn, "Cache games–bringing access-based cache attacks on aes to practice," in *2011 IEEE Symposium on Security and Privacy*. IEEE, 2011, pp. 490–505.

[25] Intel, "Intel® 64 and ia-32 architectures software developer's manual." [Online]. Available: https://cdrdv2.intel.com/v1/dl/getContent/671200

[26] Intel, "Intel® architecture memory encryption technologies." [Online]. Available: https://cdrdv2-public.intel.com/679154/multi-key-total-memory-encryption-spec-1.4.pdf

[27] intel, "Github - intel/intel-cmt-cat: User space software for intel(r) resource director technology," Apr 2025. [Online]. Available: https://github.com/intel/intel-cmt-cat/tree/master

[28] C. Intel, "Improving real-time performance by utilizing cache allocation technology," *Intel Corporation, April*, 2015.

[29] G. Irazoqui, T. Eisenbarth, and B. Sunar, "S $ a: A shared cache attack that works across cores and defies vm sandboxing–and its application to aes," in *2015 IEEE Symposium on Security and Privacy*. IEEE, 2015, pp. 591–604.

[30] M. Kayaalp, N. Abu-Ghazaleh, D. Ponomarev, and A. Jaleel, "A high-resolution side-channel attack on last-level cache," in *Proceedings of the 53rd Annual Design Automation Conference*, 2016, pp. 1–6.

[31] T. Kessous and N. Gilboa, "Prune+ plumtree-finding eviction sets at scale," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 3754–3772.

[32] V. Kiriansky, I. Lebedev, S. Amarasinghe, S. Devadas, and J. Emer, "Dawg: A defense against cache timing attacks in speculative execution processors," in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2018, pp. 974–987.

[33] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher *et al.*, "Spectre attacks: Exploiting speculative execution," *Communications of the ACM*, vol. 63, no. 7, pp. 93–101, 2020.

[34] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[35] J. Lindemann, "Faking deduplication to prevent timing side-channel attacks on memory deduplication," in *2024 Annual Computer Security Applications Conference (ACSAC)*. IEEE, 2024, pp. 380–392.

[36] F. Liu, Q. Ge, Y. Yarom, F. Mckeen, C. Rozas, G. Heiser, and R. B. Lee, "Catalyst: Defeating last-level cache side channel attacks in cloud computing," in *2016 IEEE international symposium on high performance computer architecture (HPCA)*. IEEE, 2016, pp. 406–418.

[37] F. Liu, Y. Yarom, Q. Ge, G. Heiser, and R. B. Lee, "Last-level cache side-channel attacks are practical," in *2015 IEEE symposium on security and privacy*. IEEE, 2015, pp. 605–622.

[38] J. Liu, H. Hadian, Y. Wang, D. S. Berger, M. Nguyen, X. Jian, S. H. Noh, and H. Li, "Systematic cxl memory characterization and performance analysis at scale," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025, pp. 1203–1217.

[39] A. Margaritov, D. Ustiugov, E. Bugnion, and B. Grot, "Prefetched address translation," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 1023–1036.

[40] Microsoft, "Isolation in the azure public cloud." [Online]. Available: https://learn.microsoft.com/en-us/azure/security/fundamentals/isolation-choices

[41] J. Minton and R. Balasubramonian, "flexprof: Flexible, side-channel-free memory access," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025, pp. 407–420.

[42] F. Mosquera, K. Kavi, G. Mehta, and L. K. John, "Guard cache: creating false cache hits and misses to mitigate side-channel attacks," in *2023 Silicon Valley Cybersecurity Conference (SVCC)*. IEEE, 2023, pp. 1–8.

[43] D. Ojha and S. Dwarkadas, "Timecache: using time to eliminate cache side channels when sharing software," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 375–387.

[44] Y. Oren, V. P. Kemerlis, S. Sethumadhavan, and A. D. Keromytis, "The spy in the sandbox: Practical cache attacks in javascript and their implications," in *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*, 2015, pp. 1406–1418.

[45] D. A. Osvik, A. Shamir, and E. Tromer, "Cache attacks and countermeasures: the case of aes," in *Topics in Cryptology–CT-RSA 2006: The*

Cryptographers' Track at the RSA Conference 2006, San Jose, CA, USA, February 13-17, 2005. Proceedings. Springer, 2006, pp. 1–20.

[46] C. H. Park, I. Vougioukas, A. Sandberg, and D. Black-Schaffer, "Every walk'sa hit: making page walks single-access cache hits," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 128–141.

[47] S. Park, I. Kang, Y. Moon, J. H. Ahn, and G. E. Suh, "Bcd deduplication: Effective memory compression using partial cache-line deduplication," in *Proceedings of the 26th ACM international conference on architectural support for programming languages and operating systems*, 2021, pp. 52–64.

[48] P. Pessl, D. Gruss, C. Maurice, M. Schwarz, and S. Mangard, "{DRAMA}: Exploiting {DRAM} addressing for {Cross-CPU} attacks," in *25th USENIX security symposium (USENIX security 16)*, 2016, pp. 565–581.

[49] pmem.io, "Using the memmap kernel option — persistent memory documentation," Oct 2019. [Online]. Available: https://docs.pmem.io/persistent-memory/getting-started-guide/creating-development-environments/linux-environments/linux-memmap

[50] M. Poess and C. Floyd, "New tpc benchmarks for decision support and web commerce," *SIGMOD Rec.*, vol. 29, no. 4, p. 64–71, Dec. 2000. [Online]. Available: https://doi.org/10.1145/369275.369291

[51] A. Purnal, F. Turan, and I. Verbauwhede, "Prime+ scope: Overcoming the observer effect for high-precision cache contention attacks," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 2906–2920.

[52] K. Ramkrishnan, S. McCamant, P. C. Yew, and A. Zhai, "First time miss: Low overhead mitigation for shared memory cache side channels," in *Proceedings of the 49th International Conference on Parallel Processing*, 2020, pp. 1–11.

[53] I. Red Hat, "What is a linux container?" https://www.redhat.com/en/topics/containers/whats-a-linux-container, 2025, accessed: 2025-Nov.

[54] *Red Hat Enterprise Linux 6: Virtualization Administration Guide*, Red Hat, Inc., 2017, accessed: 2025-Nov. [Online]. Available: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/6/html/virtualization_administration_guide/chap-virtualization_administration_guideadvanced

[55] R. Rosen, "Namespaces and cgroups, the basis of linux containers," *Seville, Spain, Feb*, 2016.

[56] J. H. Ryoo, N. Gulur, S. Song, and L. K. John, "Rethinking tlb designs in virtualized environments: A very large part-of-memory tlb," *ACM SIGARCH Computer Architecture News*, vol. 45, no. 2, pp. 469–480, 2017.

[57] G. Saileshwar and M. K. Qureshi, "Lookout for zombies: Mitigating flush+ reload attack on shared caches by monitoring invalidated lines," *arXiv preprint arXiv:1906.02362*, 2019.

[58] A. Shafiee, A. Gundu, M. Shevgoor, R. Balasubramonian, and M. Tiwari, "Avoiding information leakage in the memory controller with fixed service policies," in *Proceedings of the 48th International Symposium on Microarchitecture*, 2015, pp. 89–101.

[59] J. P. Stevenson, *Fine-grain in-memory deduplication for large-scale workloads*. Stanford University, 2013.

[60] Synopsys, Inc., "Silicon to systems design solutions," https://www.synopsys.com.

[61] S. Van Schaik, C. Giuffrida, H. Bos, and K. Razavi, "Malicious management unit: Why stopping cache attacks in software is harder than you think," in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 937–954.

[62] P. Vila, B. Köpf, and J. F. Morales, "Theory and practice of finding eviction sets," in *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2019, pp. 39–54.

[63] Y. Wang, A. Ferraiuolo, and G. E. Suh, "Timing channel protection for a shared memory controller," in *2014 IEEE 20th International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2014, pp. 225–236.

[64] M. Werner, T. Unterluggauer, L. Giner, M. Schwarz, D. Gruss, and S. Mangard, "{ScatterCache}: thwarting cache attacks via cache set randomization," in *28th USENIX Security Symposium (USENIX Security 19)*, 2019, pp. 675–692.

[65] Y. Yarom and K. Falkner, "{FLUSH+ RELOAD}: A high resolution, low noise, l3 cache {Side-Channel} attack," in *23rd USENIX security symposium (USENIX security 14)*, 2014, pp. 719–732.

[66] H. Yavarzadeh, A. Agarwal, M. Christman, C. Garman, D. Genkin, A. Kwong, D. Moghimi, D. Stefan, K. Taram, and D. Tullsen, "Pathfinder: High-resolution control-flow attacks exploiting the conditional branch predictor," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 770–784.

[67] Y. Zhang, A. Juels, M. K. Reiter, and T. Ristenpart, "Cross-tenant side-channel attacks in paas clouds," in *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, 2014, pp. 990–1003.

[68] Z. N. Zhao, A. Morrison, C. W. Fletcher, and J. Torrellas, "Last-level cache side-channel attacks are feasible in the modern public cloud," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, pp. 582–600.

[69] Y. Zhu, B. Chen, Z. N. Zhao, and C. W. Fletcher, "Controlled preemption: Amplifying side-channel attacks from userspace," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025, pp. 162–177.